

DATABASE DESIGN & IMPLEMENTATION DOCUMENT



Not just NAILS.. IT's the WHOLE MOOD
Online Nail Art & Nail Beauty Online Platform

WEEK - 2

2. Service and Category Management System

Scenario

Nailverse wants to manage information about the nail services it provides and the categories those services fall under. Each service belongs to a specific category (e.g. Manicure, Pedicure, Nail Art, Extensions), and the system should maintain service details such as name, price, duration, and slot availability. This continues directly from the Service and Category attributes outlined in the Week 1 entity list.

1. Design Service and Category Tables

Category Table:

```
SQL> CREATE TABLE Category(Category_ID VARCHAR2(7),  
2 Category_Name VARCHAR2(30),  
3 Description VARCHAR2(100));
```

Table created.

Service Table:

```
SQL> CREATE TABLE Service(Service_ID VARCHAR2(7),  
2 Service_Name VARCHAR2(50),  
3 Category_ID VARCHAR2(7),  
4 Price NUMBER(8,2),  
5 Duration_Minutes NUMBER,  
6 Slots_Available NUMBER);
```

Table created.

The Category and Service tables have now been created.
Next, we need to identify the relationship between the tables and apply the required constraints.

2. Define Relationships and Constraints

Relationships:

A single Category can contain multiple Services.

The relationship between Category and Service is One to many (1 -> *)

After identifying the relationship, we need to determine which key will enforce it. Before proceeding, we need to understand KEY constraints

- **PRIMARY KEY:** The column or field that uniquely identifies a row or tuple. Which field can serve as the Primary Key in these two tables? In the Category Table, Category_ID is a unique value used to identify each category. Similarly, in the Service Table, Service_ID is a unique column.

- **FOREIGN KEY:** The key that connects two tables. Therefore, Category_ID acts as a Foreign Key in the Service Table. We can also consider other constraints such as NOT NULL, CHECK and UNIQUE.

- **NOT NULL:** Ensures that a column cannot contain an empty (null) value. For example, a Service name or Category name cannot be empty. Each category or service must have a name.
- **CHECK:** Checks that the values in a column meet a specified condition. For example, the Price of a Service should always be a positive number, and Slots_Available cannot be negative.
- **UNIQUE:** Ensures that a column does not contain repeated values. For example, if we apply a UNIQUE constraint to Category_Name, no two categories can have the same name.

Defining Relationship and Creating Constraints for Service and Category:

Primary Key:

```
SQL> ALTER TABLE Category
2 ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);
```

Table altered.

```
SQL> ALTER TABLE Service
2 ADD CONSTRAINT PK_Service PRIMARY KEY (Service_ID);
```

Table altered.

Creating Relationship using FOREIGN KEY:

```
SQL> ALTER TABLE Service
2 ADD CONSTRAINT FK_Service_Category
3 FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);
```

Table altered.

Adding Constraints:

NOT NULL:

```
SQL> ALTER TABLE Category
2 MODIFY Category_Name VARCHAR2(30) NOT NULL;
```

```
Table altered.
```

```
SQL> ALTER TABLE Service  
2 MODIFY Service_Name VARCHAR2(50) NOT NULL;
```

```
Table altered.
```

CHECK:

```
SQL> ALTER TABLE Service  
2 ADD CONSTRAINT CHK_Price CHECK (Price > 0);
```

```
Table altered.
```

```
SQL> ALTER TABLE Service  
2 ADD CONSTRAINT CHK_Slots CHECK (Slots_Available >= 0);
```

```
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category  
2 ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);
```

```
Table altered.
```

3. Insert Values

Category Table:

```
SQL> INSERT INTO Category  
2 VALUES (  
3 '&Category_ID',  
4 '&Category_Name',  
5 '&Description');  
Enter value for category_id: Cat0001  
old 3: '&Category_ID',  
new 3: 'Cat0001',  
Enter value for category_name: Manicure  
old 4: '&Category_Name',  
new 4: 'Manicure',  
Enter value for description: Hand and nail care services  
old 5: '&Description',  
new 5: 'Hand and nail care services')
```

```
1 row created.
```

```
SQL> /  
Enter value for category_id: Cat0002  
old 3: '&Category_ID',  
new 3: 'Cat0002',  
Enter value for category_name: Pedicure  
old 4: '&Category_Name',  
new 4: 'Pedicure',  
Enter value for description: Foot and nail care services  
old 5: '&Description',  
new 5: 'Foot and nail care services')
```

```
1 row created.
```

Like this, insert the remaining categories: Nail Art, Extensions.

Service Table:

```

SQL> INSERT INTO Service
  2  VALUES(
  3  '&Service_ID',
  4  '&Service_Name',
  5  '&Category_ID',
  6  &Price,
  7  &Duration_Minutes,
  8  &Slots_Available);
Enter value for service_id: SRV001
old 3: '&Service_ID',
new 3: 'SRV001',
Enter value for service_name: Classic Manicure
old 4: '&Service_Name',
new 4: 'Classic Manicure',
Enter value for category_id: Cat0001
old 5: '&Category_ID',
new 5: 'Cat0001',
Enter value for price: 499
old 6: &Price,
new 6: 499,
Enter value for duration_minutes: 45
old 7: &Duration_Minutes,
new 7: 45,
Enter value for slots_available: 10
old 8: &Slots_Available)
new 8: 10)

1 row created.

```

Like this insert few more rows.

After Insertion – Category Table:

```

SQL> SELECT * FROM Category;

CATEGOR CATEGORY_NAME
-----
DESCRIPTION
-----
Cat0001 Manicure
Hand and nail care services

Cat0002 Pedicure
Foot and nail care services

Cat0003 Nail Art
Creative nail designs and decorations

Cat0004 Extensions
Nail extension and enhancement services

4 rows selected.

```

After Insertion – Service Table:

```

SQL> SELECT * FROM Service;

SERVICE
-----
SERVICE_NAME
-----
CATEGOR      PRICE DURATION_MINUTES SLOTS_AVAILABLE

```

```

-----
SRV001
Classic Manicure
Cat0001      499          45          10

SRV002
Gel Manicure
Cat0001      799          60           8

SRV003
Spa Manicure
Cat0001      999          75           5

SRV004
Classic Pedicure
Cat0002      599          45          10

SRV005
Spa Pedicure
Cat0002     1099          90           6

SRV006
3D Nail Art
Cat0003     1499          90           4

SRV007
French Tip Art
Cat0003      699          45           8

SRV008
Acrylic Extension
Cat0004     1899         120           3

8 rows selected.

```

CRUD Operations:

Insert Record:

```

SQL> INSERT INTO Service (Service_ID, Service_Name, Category_ID, Price, Duration_Minutes,
2  Slots_Available)
   2  VALUES ('SRV009','Chrome Nail Art', 'Cat0003', 1299, 80, 5);

1 row created.

```

Update Price:

```

SQL> UPDATE Service SET Price = 650 WHERE Service_Name = 'Classic Pedicure';

1 row updated.

```

Delete a Service:

```

SQL> DELETE FROM Service WHERE Service_Name = 'French Tip Art';

1 row deleted.

```

5. Generate Category-wise Service Reports

Write SQL queries to:

- Display all services under each category.

```

SQL> SELECT c.Category_Name, s.Service_Name, s.Price, s.Duration_Minutes
2  FROM Service s JOIN Category c ON s.Category_ID = c.Category_ID;

```

CATEGORY_NAME	SERVICE_NAME	PRICE	DURATION_MINUTES
Manicure	Classic Manicure	499	45
Manicure	Gel Manicure	799	60
Manicure	Spa Manicure	999	75
Pedicure	Classic Pedicure	650	45
Pedicure	Spa Pedicure	1099	90
Nail Art	3D Nail Art	1499	90
Nail Art	Chrome Nail Art	1299	80
Extensions	Acrylic Extension	1899	120

8 rows selected.

- Count number of services in each category.

```
SQL> SELECT c.Category_Name, COUNT(s.Service_ID) AS Service_Count
  2   FROM Service s JOIN Category c ON s.Category_ID = c.Category_ID
  3   GROUP BY c.Category_Name;
```

CATEGORY_NAME	SERVICE_COUNT
Manicure	3
Pedicure	2
Nail Art	2
Extensions	1

- Find highest-priced service in each category.

```
SQL> SELECT c.Category_Name, s.Service_Name, s.Price AS Highest_Price
  2   FROM Service s
  3   JOIN Category c ON s.Category_ID = c.Category_ID
  4   WHERE s.Price = (
  5     SELECT MAX(s2.Price)
  6     FROM Service s2
  7     WHERE s2.Category_ID = s.Category_ID
  8   );
```

CATEGORY_NAME	SERVICE_NAME	HIGHEST_PRICE
Manicure	Spa Manicure	999
Pedicure	Spa Pedicure	1099
Nail Art	3D Nail Art	1499
Extensions	Acrylic Extension	1899

- Display categories having more than 2 services.

```
SQL> SELECT c.Category_Name
  2   FROM Service s JOIN Category c ON s.Category_ID = c.Category_ID
  3   GROUP BY c.Category_Name HAVING COUNT(s.Service_ID) > 2;
```

CATEGORY_NAME
Manicure

- Find average service price category-wise.

```
SQL> SELECT c.Category_Name, AVG(s.Price) AS Avg_Price
  2   FROM Service s JOIN Category c ON s.Category_ID = c.Category_ID
  3   GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVG_PRICE
---------------	-----------

Manicure	765.6667
Pedicure	874.5
Nail Art	1399
Extensions	1899